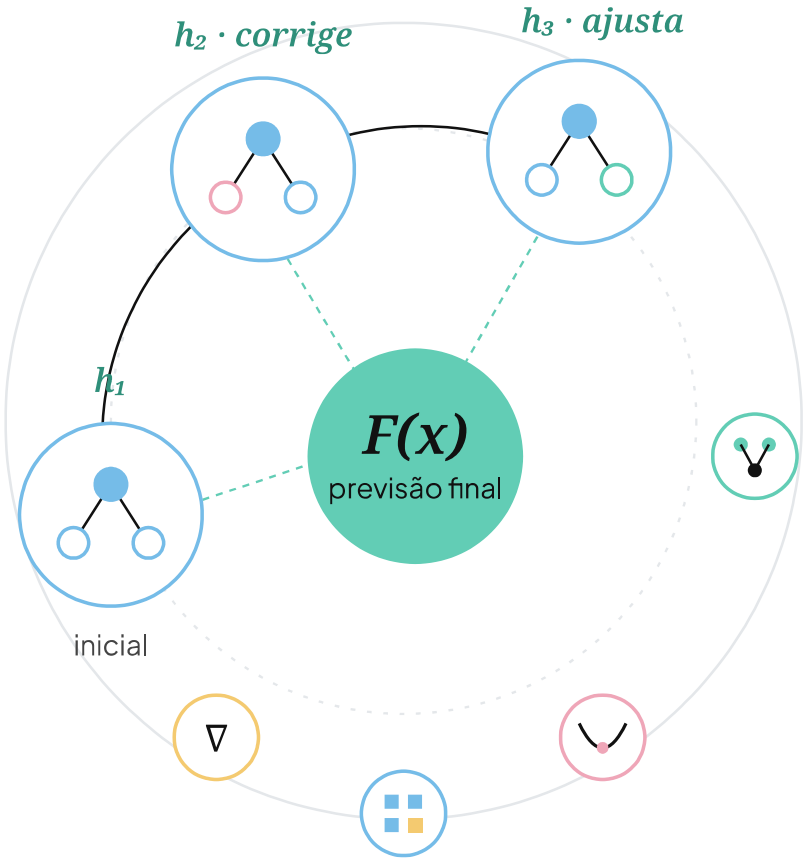
AULA 7.4 · MÓDULO 7

BOOSTING

XGBOOST, LIGHTGBM E CATBOOST

Como modelos aprendem sequencialmente para melhorar suas previsões.

Algoritmos Modernos e Ajuste Fino · 2 horas



AULA 7.4

Roteiro da aula

01 · 10 min

Revisão de Bagging

Ensembles, bootstrap e Random Forest

02 · 20 min

Boosting e resíduos

Aprendizado sequencial com exemplo numérico

03 · 15 min

Gradient Boosting

Gradientes, learning rate, árvores e profundidade

04 · 15 min

XGBoost

Objetivo regularizado, gradiente e Hessiana

05 · 15 min

LightGBM

Histogramas, leaf-wise, GOSS e EFB

06 · 15 min

CatBoost

Categorias, estatísticas ordenadas e Ordered Boosting

07 · 25 min

Benchmark guiado

Três algoritmos no mesmo split, do notebook vazio

08 · 5 min

Quiz e conclusão

Erros comuns e desafio final

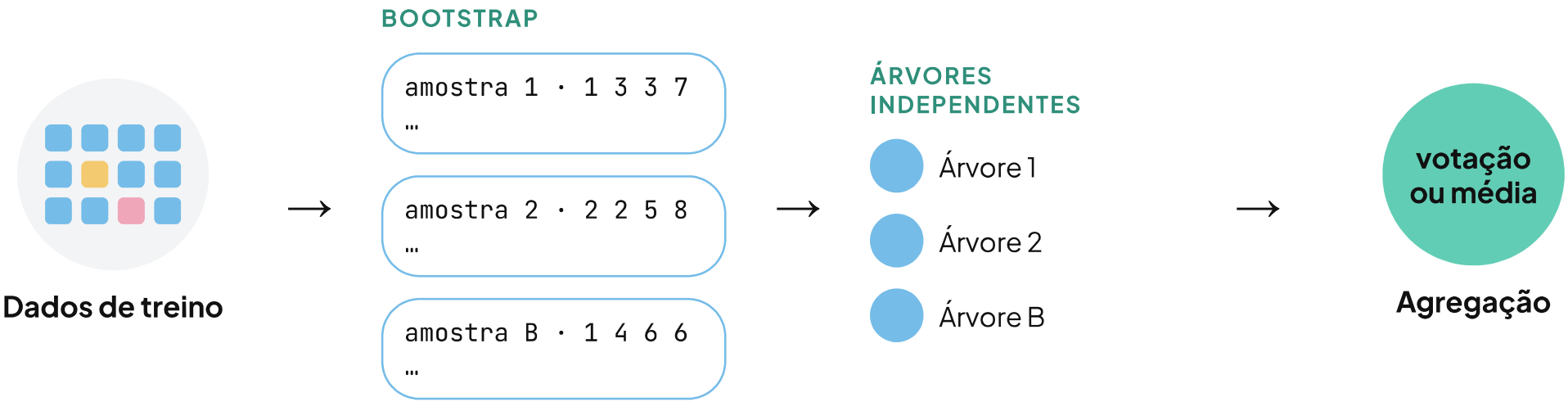
01

Recapitulação

Ensemble Learning, Bagging, Bootstrap e Random Forest

01 · RECAPITULAÇÃO

Bagging e Random Forest



Diversidade: amostras e características sorteadas

Treinamento: árvores não dependem umas das outras

Efeito: redução de variância

A PERGUNTA DESTA AULA

E se, em vez de treinar diversas árvores independentemente, treinássemos uma nova árvore para melhorar o modelo que já existe?



02

Boosting e resíduos

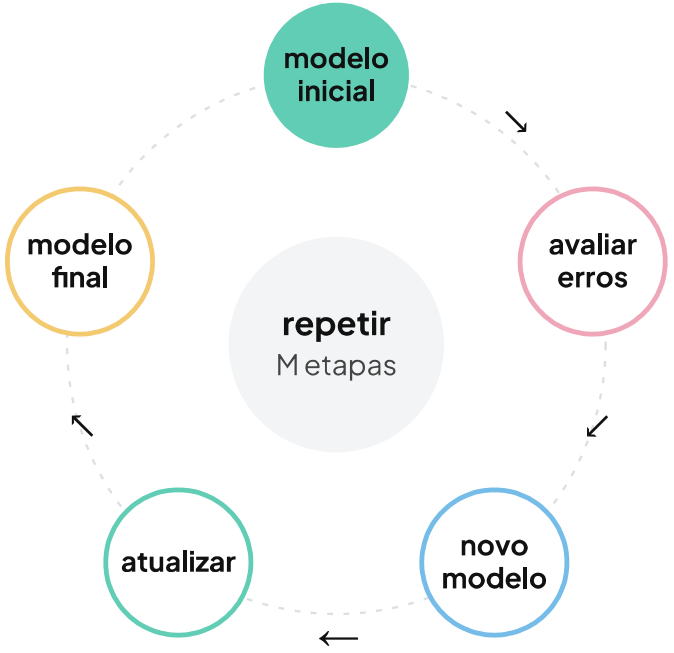
Aprendizado sequencial, Bagging vs. Boosting e um exemplo resolvido à mão

02 · BOOSTING

O que é Boosting

Boosting é uma família de métodos de Ensemble Learning em que novos modelos são adicionados sequencialmente para melhorar o conjunto de previsões construído até aquele momento.

Cada novo modelo depende do resultado do conjunto anterior. No Bagging, os modelos são treinados de forma relativamente independente.



02 · BOOSTING

Vocabulário do Boosting

- **Modelo fraco**
  Pouco melhor que um palpite quando usado sozinho

- **Aprendizado sequencial**
  A etapa m usa o resultado das etapas 1 até m – 1

- **Atualização**
  Previsão nova = previsão atual + contribuição da nova árvore

- **Modelo final**
  Modelo inicial somado às contribuições de todas as árvores

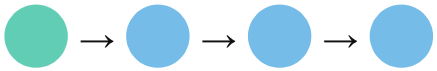
- **Modelo base**
  Tipo de modelo repetido no ensemble, aqui árvores de decisão

- **Ensemble**
  Conjunto de todos os modelos adicionados até o momento

- **Correção progressiva**
  Cada etapa reduz um pouco a perda, sem resolver tudo de uma vez

02 · BOOSTING

Bagging vs. Boosting

	Bagging 	Boosting 
Dados por modelo	Amostras bootstrap	Dados completos ou subamostras, com alvos atualizados a cada etapa
Treinamento	Relativamente independente	Sequencial: considera o ensemble atual
Paralelismo	Entre árvores	Dentro de cada árvore; as árvores seguem em ordem
Hiperparâmetros	Valores padrão costumam funcionar	Mais sensível: taxa, nº de árvores, profundidade
Efeito principal	Reduz variância	Reduz viés; pode sobreajustar sem controle
Exemplo	Random Forest	Gradient Boosting

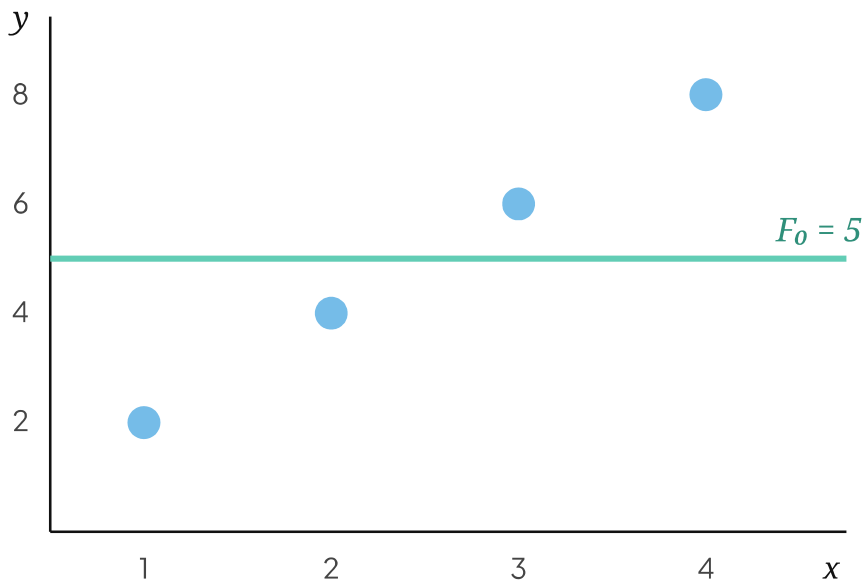
02 · EXEMPLO NUMÉRICO · PASSO 1

Dados e modelo inicial

x	y real	$F_0(x)$
1	2	**5**
2	4	**5**
3	6	**5**
4	8	**5**

$$F_0(x) = \bar{y} = (2 + 4 + 6 + 8) / 4 = 5$$

Para a perda quadrática, a média é a constante que minimiza a soma dos erros ao quadrado.



02 · EXEMPLO NUMÉRICO · PASSO 2

Resíduos do modelo inicial

Perda quadrática

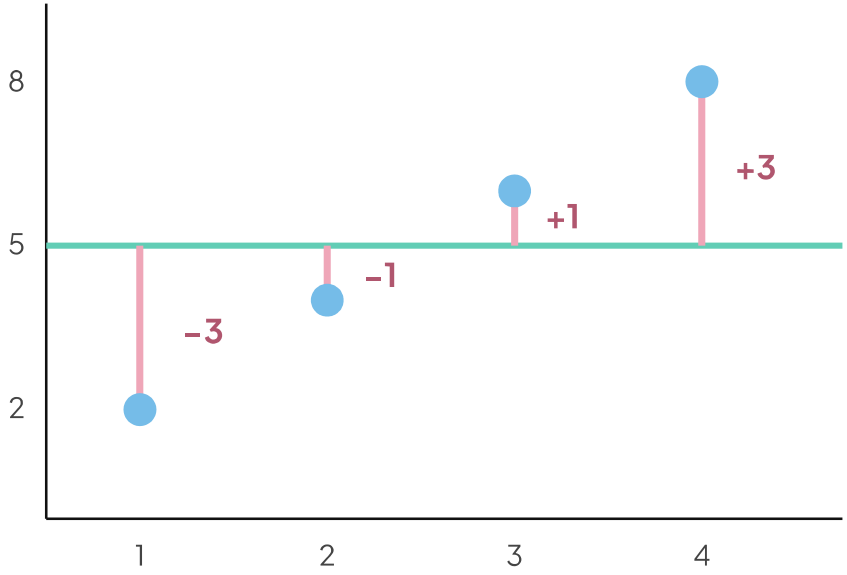
$$L = \tfrac{1}{2}(y - F)^2 \Rightarrow -\partial L/\partial F = y - F$$

Resíduo de cada amostra

$$r_i = y_i - F_0(x_i)$$

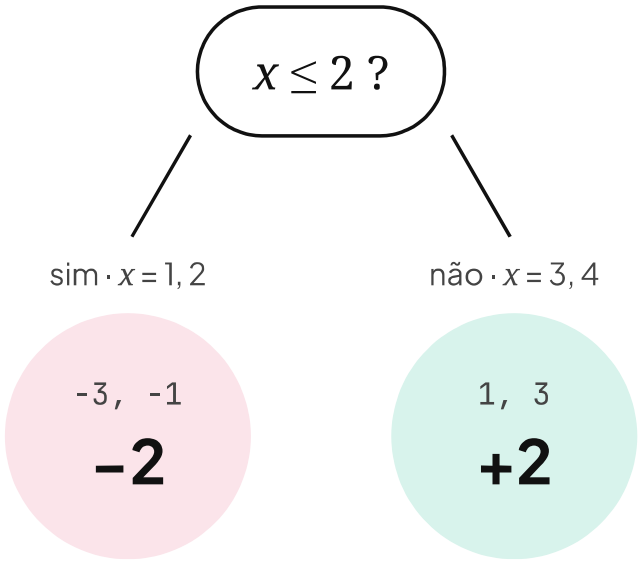
`2-5 = -3` `4-5 = -1` `6-5 = 1` `8-5 = 3`

Soma dos quadrados: 9 + 1 + 1 + 9 = **20**



02 · EXEMPLO NUMÉRICO · PASSO 3

Primeira árvore sobre os resíduos



1. O alvo da árvore são os **resíduos**, não o y original.
2. A divisão $x \leq 2$ separa resíduos negativos de positivos.
3. Cada folha prevê a **média dos resíduos** que caem nela.

$$h_1(x) = -2 \text{ se } x \leq 2;\ +2 \text{ caso contrário}$$

02 · EXEMPLO NUMÉRICO · PASSO 4

Atualização do ensemble

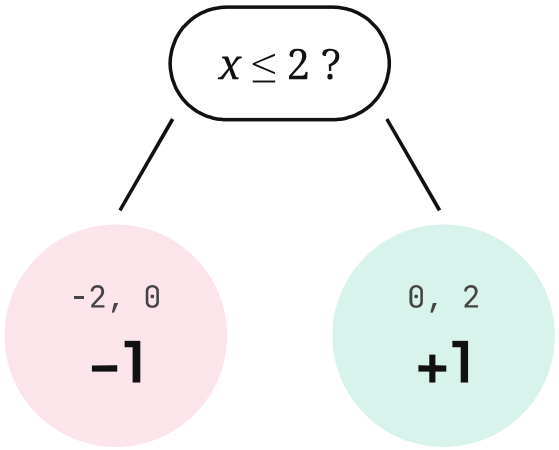
$$F_1(x) = F_0(x) + \eta \cdot h_1(x)$$ $\eta = 0{,}5$

x	y	$h_1(x)$	$F_0 + \eta \cdot h_1$	$F_1(x)$	novo resíduo
1	2	−2	`5 + 0,5·(-2)`	**4**	**−2**
2	4	−2	`5 + 0,5·(-2)`	**4**	**0**
3	6	+2	`5 + 0,5·(+2)`	**6**	**0**
4	8	+2	`5 + 0,5·(+2)`	**6**	**2**

Previsões: **[4, 4, 6, 6]** Resíduos: **[−2, 0, 0, 2]** Soma dos quadrados: 20 → **8**

02 · EXEMPLO NUMÉRICO · PASSO 5

Segunda árvore e nova correção



$$F_2(x) = F_1(x) + 0{,}5 \cdot h_2(x)$$

F_1 = [4, 4, 6, 6]

F_2 = [3,5; 3,5; 6,5; 6,5]

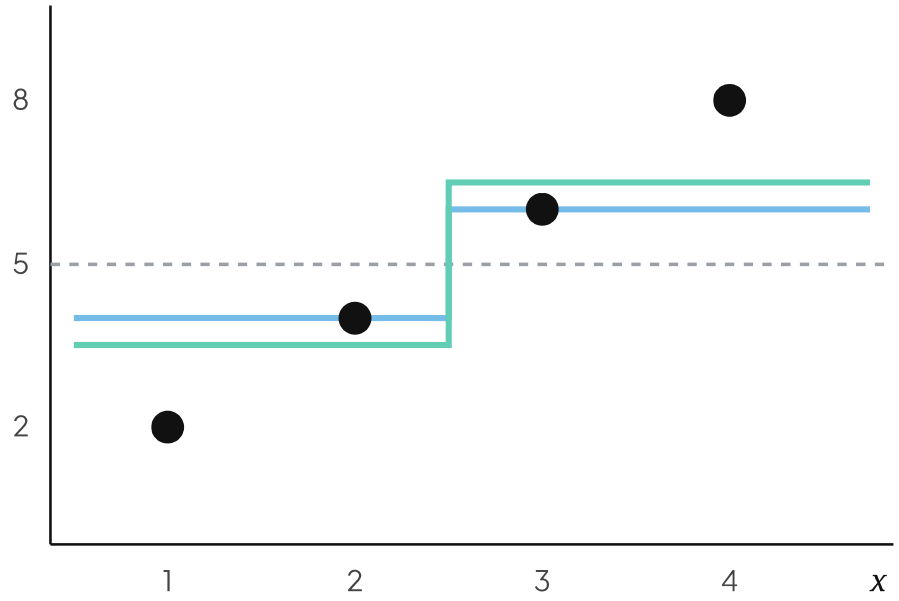
r = [-1,5; 0,5; -0,5; 1,5]

Soma dos quadrados: 8 → **5**

Com um único corte, os pontos do mesmo grupo recebem a mesma correção. Árvores com mais divisões permitem correções mais finas.

02 · EXEMPLO NUMÉRICO · RESUMO

Evolução da previsão do ensemble



F_0 [5, 5, 5, 5] · SSE 20

F_1 [4, 4, 6, 6] · SSE 8

F_2 [3,5; 3,5; 6,5; 6,5] · SSE 5

valores reais y

Demonstração didática com perda quadrática e folhas constantes. Não representa a implementação completa de XGBoost ou CatBoost.

02 · DA INTUIÇÃO À MATEMÁTICA

Resíduo e gradiente negativo

$$r_{im} = -\left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}\right]_{F = F_{m-1}}$$

i	amostra do conjunto de treino
m	etapa do Boosting
L	função de perda
y_i	valor real
$F(x_i)$	previsão atual do ensemble
$-\partial$	direção que reduz a perda

Resíduo

$y - F$. Coincide com o gradiente negativo na perda quadrática $\frac{1}{2}(y - F)^2$.

Pseudo-resíduo

Gradiente negativo de qualquer perda diferenciável. É o alvo que a nova árvore aproxima.

03

Gradient Boosting

Modelo aditivo, classificação, learning rate, número de árvores e profundidade

03 · GRADIENT BOOSTING

Modelo aditivo e função objetivo

$$F_m(x) = F_{m-1}(x) + \eta\, h_m(x)$$

F_m
modelo após a etapa *m*

F_{m-1}
modelo anterior, mantido fixo

η
taxa de aprendizado, encolhe a correção

h_m
nova árvore, ajustada aos pseudo-resíduos

$$\min \sum_{i=1}^{n} L(y_i, F(x_i))$$

O objetivo é minimizar a perda total nos dados de treino. Cada etapa dá um passo de gradiente descendente no espaço de funções.

03 · GRADIENT BOOSTING

Algoritmo passo a passo

1	$F_0(x) = \text{argmin}_c \, \Sigma_i \, L(y_i, c)$	Constante inicial: média ou log das chances
2a	$r_{im} = -\partial L(y_i, F)/\partial F \, \vert_{F=F_{m-1}}$	Pseudo-resíduos de cada amostra
2b	$h_m \leftarrow$ árvore em (x_i, r_{im})	Árvore de regressão aproxima os pseudo-resíduos
2c	$w_j = \text{argmin}_w \, \Sigma_{i \in \text{folha } j} \, L(y_i, F_{m-1} + w)$	Valor ótimo de cada folha
2d	$F_m = F_{m-1} + \eta \, h_m$	Atualização; repetir 2a–2d para $m = 1 \ldots M$
3	$F_M(x)$	Modelo final: soma de todas as contribuições

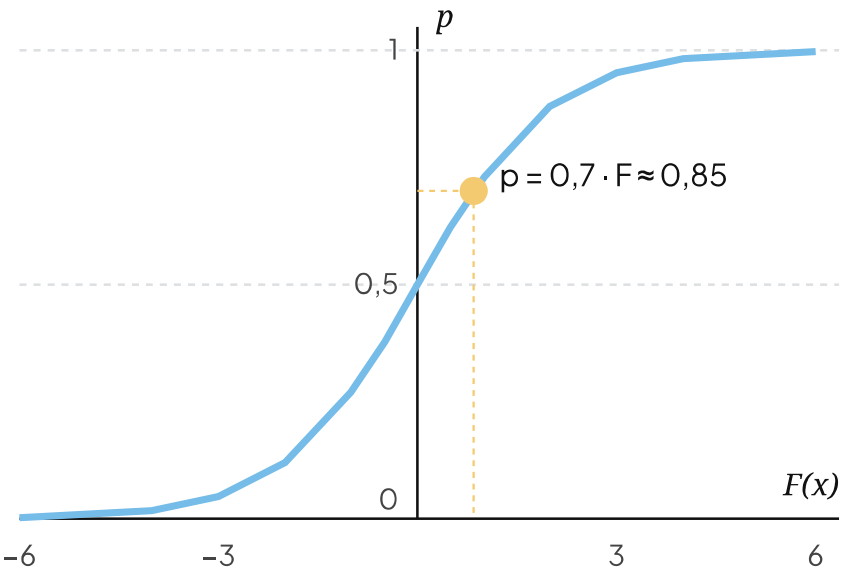
03 · CLASSIFICAÇÃO BINÁRIA

Do escore à probabilidade

$$p = \frac{1}{1 + e^{-F(x)}}$$

$F(x)$	escore bruto ou logit, soma das árvores
p	probabilidade prevista da classe 1
logit	$F = \log(p / (1 - p))$, o log das chances

As árvores somam correções no espaço do logit. A conversão para probabilidade acontece no final.



03 · CLASSIFICAÇÃO BINÁRIA

Pseudo-resíduo da Binary Cross-Entropy

$$L = -[\, y \ln p + (1 - y) \ln(1 - p) \,]$$

$$\partial L/\partial p = -y/p + (1 - y)/(1 - p)$$

$$\partial p/\partial F = p(1 - p)$$

$$\partial L/\partial F = -y(1 - p) + (1 - y)p = p - y$$

$$r_i = y_i - p_i$$

+0,3

$y = 1, p = 0{,}7$

Correção moderada para cima: aumentar o logit.

−0,7

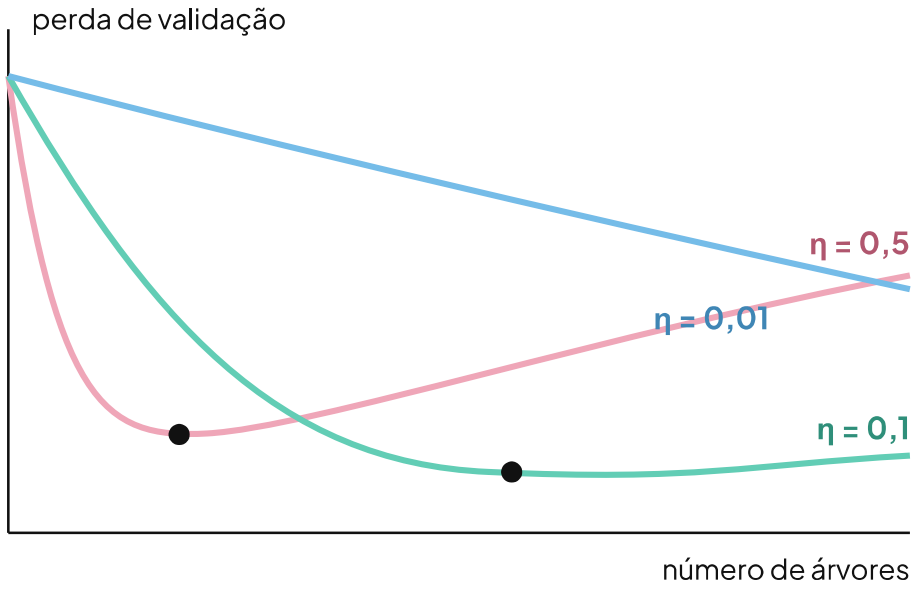
$y = 0, p = 0{,}7$

Modelo confiante e errado: correção forte para baixo.

Esta forma vale para a BCE parametrizada pelo logit. Outras perdas geram outros pseudo-resíduos.

03 · HIPERPARÂMETROS CENTRAIS

Learning rate



Curvas conceituais. Curvas reais dependem de dados e parâmetros.

$$F_m = F_{m-1} + \eta\, h_m$$

η pequeno

Correções menores. Exige mais árvores e mais tempo.

η grande

Atualizações maiores. Aprende rápido, mas pode prejudicar a generalização.

Nenhuma taxa é sempre a melhor: escolha por validação.

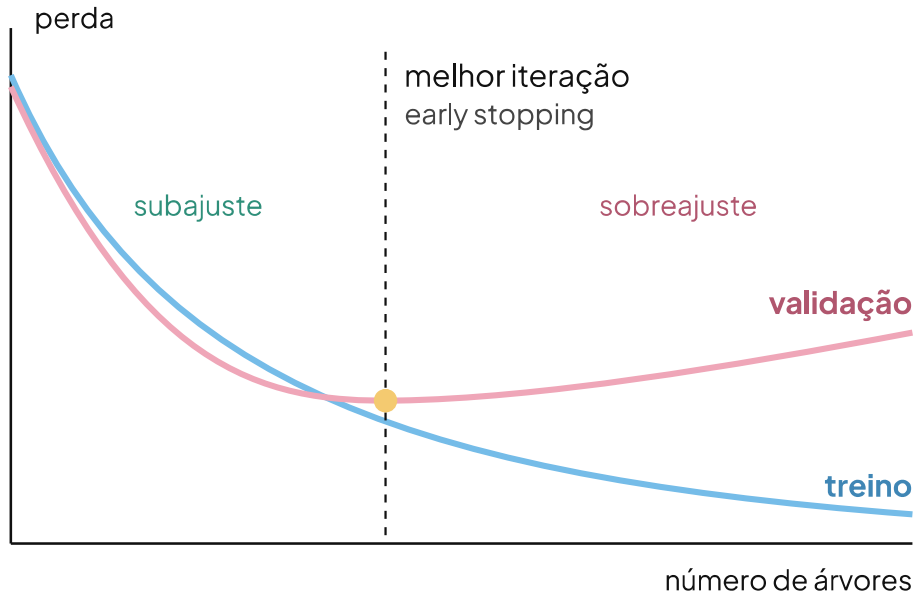
03 · HIPERPARÂMETROS CENTRAIS

Número de estimadores

`n_estimators = M`

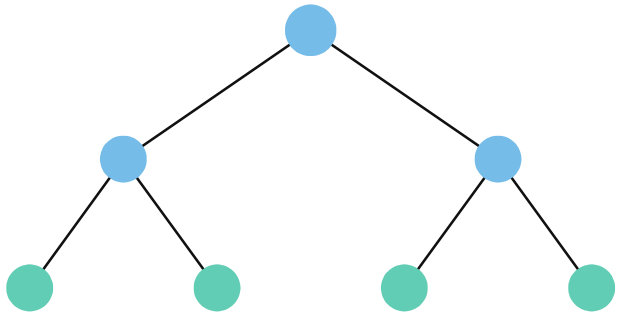
Capacidade	mais árvores, mais ajuste ao treino
Learning rate	η menor pede mais árvores
Tempo	cresce com o número de árvores
Overfitting	validação piora após o mínimo

Early stopping interrompe quando a métrica de validação para de melhorar por um número fixo de rodadas.



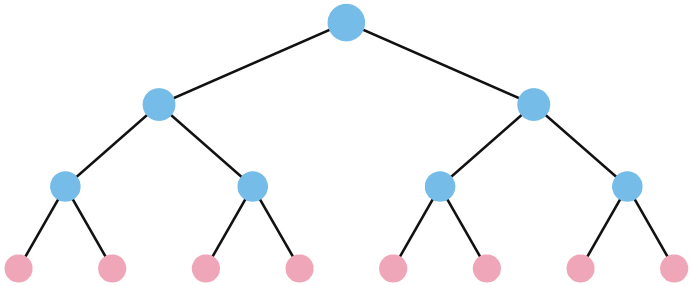
03 · HIPERPARÂMETROS CENTRAIS

Profundidade e complexidade



Árvore rasa · profundidade 2

Até 4 folhas · interações de até 2 variáveis · viés maior, variância menor



··· continua até 2^d folhas

Árvore profunda · profundidade d

Até 2^d folhas · interações complexas · maior risco de ajustar ruído

---

No Boosting, cada árvore faz uma pequena correção e a complexidade vem da soma. Por isso são comuns árvores relativamente pequenas.

04

XGBoost

Objetivo regularizado, gradiente e Hessiana, hiperparâmetros

04 · XGBOOST

O que é XGBoost

HERDADO DO GRADIENT BOOSTING

Treinamento sequencial
árvores somadas em etapas

Funções de perda
logística, quadrática, ranking...

Gradientes
direção que reduz a perda



O QUE ACRESCENTA

Informação de segunda ordem
gradiente e Hessiana

Regularização no objetivo
penaliza folhas e pesos

Valores ausentes
direção padrão aprendida por divisão

Otimizações computacionais
histogramas, paralelismo interno

04 · XGBOOST

Objetivo regularizado

$$Obj = \text{Loss} + \Omega$$ erro de previsão + penalização de complexidade

$$\Omega(f) = \gamma T + \tfrac{1}{2}\lambda \sum_{j=1}^{T} w_j^2 + \alpha \sum_{j=1}^{T} |w_j|$$

T
número de folhas da árvore

w_j
peso da folha *j*, valor somado ao escore

γ
custo de cada folha adicional · `gamma`

λ
regularização L2 dos pesos · `reg_lambda`

α
regularização L1 dos pesos · `reg_alpha`

Atenção O `gamma` do XGBoost não é o `gamma` do kernel RBF da SVM (Aula 7.2).

04 · XGBOOST

Gradiente e Hessiana

$$Obj^{(t)} \approx \Sigma_i \left[g_i f_t(x_i) + \tfrac{1}{2} h_i f_t(x_i)^2 \right] + \Omega(f_t)$$

g_i	$\partial L/\partial F$: gradiente da perda na previsão atual
h_i	$\partial^2 L/\partial F^2$: Hessiana escalar, curvatura da perda
f_t	nova árvore adicionada na etapa t
Ω	regularização da árvore

Peso ótimo da folha j (com $\alpha = 0$)

$$w_j^* = -G_j / (H_j + \lambda)$$

G_j, H_j: somas de g_i e h_i na folha

EXEMPLO · FOLHA x ≤ 2, $F_0 = 5$

Perda quadrática: $g = F - y$, $h = 1$

```
g = [3, 1] → G = 4, H = 2
λ = 0 → w* = -4/2 = -2
λ = 1 → w* = -4/3 ≈ -1,33
```

Logística: $g = p - y$, $h = p(1 - p)$

04 · XGBOOST

Ganho de uma divisão

$$\text{Ganho} = \tfrac{1}{2}\left[\frac{G_L^{\,2}}{H_L + \lambda} + \frac{G_R^{\,2}}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda}\right] - \gamma$$

Esquerda e direita: melhoria de cada folha nova

Último termo: valor do nó sem dividir

– γ: a divisão só acontece se o ganho superar γ

EXEMPLO · x ≤ 2, λ = 1

```
G_L = 4, H_L = 2
G_R = -4, H_R = 2
½ (16/3 + 16/3 - 0) ≈ 5,33
Ganho ≈ 5,33 - γ
```

Hiperparâmetros do XGBoost

Parâmetro	Significado / controla	Ao aumentar	Ao diminuir	Overfitting ↑	Tempo ↑
`n_estimators`	máximo de árvores	mais capacidade	pode subajustar	aumenta risco	sim, linear
`learning_rate`	η, encolhe cada árvore	passos maiores	exige mais árvores	aumenta risco	tende a cair
`max_depth`	profundidade máxima	interações complexas	árvores simples	aumenta risco	sim
`min_child_weight`	soma mínima de h_j na folha	mais conservador	folhas pequenas	reduz risco	tende a cair
`gamma`	ganho mínimo para dividir	poda divisões fracas	mais divisões	reduz risco	tende a cair
`subsample`	fração de linhas por árvore	menos aleatoriedade	mais aleatoriedade	aumenta risco	sim

Overfitting ↑ e Tempo ↑ descrevem o efeito típico de aumentar o parâmetro. Efeitos reais dependem dos dados.

Hiperparâmetros do XGBoost

Parâmetro	Significado / controla	Ao aumentar	Ao diminuir	Overfitting ↑	Tempo ↑
`colsample_bytree`	fração de colunas por árvore	árvores parecidas	mais diversidade	aumenta risco	sim
`reg_alpha`	L1 nos pesos, α	zera pesos pequenos	menos penalização	reduz risco	pouco efeito
`reg_lambda`	L2 nos pesos, λ	encolhe pesos	correções maiores	reduz risco	pouco efeito
`objective`	função de perda; define g e h	—	—	—	depende
`eval_metric`	métrica monitorada na validação	—	—	—	pequeno custo
`early_stopping_rounds`	rodadas sem melhoria antes de parar	mais paciência	para mais cedo	pouco efeito*	sim

* O modelo final usa a melhor iteração; mais paciência apenas gasta mais rodadas. No XGBoost ≥ 1.6, `early_stopping_rounds` vai no construtor.

04 · XGBOOST · CÓDIGO

XGBClassifier em poucas linhas

```
from xgboost import XGBClassifier

modelo = XGBClassifier(
    n_estimators=800,          #  M máximo
    learning_rate=0.05,        #  η
    max_depth=5,
    min_child_weight=1,        #  soma mínima de h
    gamma=0.0,                 #  ganho mínimo
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,            #  λ
    objective="binary:logistic",
    eval_metric="auc",
    early_stopping_rounds=40,
)
modelo.fit(X_train, y_train,
           eval_set=[(X_val, y_val)], verbose=False)
print(modelo.best_iteration)
```

eval_set

Conjunto de validação monitorado a cada rodada.

early_stopping_rounds

No construtor a partir do XGBoost 1.6; recomendado na série 2.x.

best_iteration

Rodada com melhor métrica de validação.

O conjunto de teste nunca entra no `eval_set`.

05

LightGBM

Histogramas, crescimento leaf-wise, num_leaves, GOSS e EFB

05 · LIGHTGBM

O que é LightGBM

Light Gradient Boosting Machine

Implementação de Gradient Boosting focada em eficiência e velocidade, especialmente em dados tabulares.



Histogramas

valores agrupados em bins



Leaf-wise

expande a folha de maior ganho



Divisões

candidatas nas fronteiras dos bins



Eficiência

menos memória e menos candidatos



num_leaves

controle principal de complexidade



Categóricas

suporte próprio, se declaradas

Não é sempre mais rápido que o XGBoost: o desempenho depende do tamanho dos dados, dos parâmetros, do hardware e da configuração.

05 · LIGHTGBM

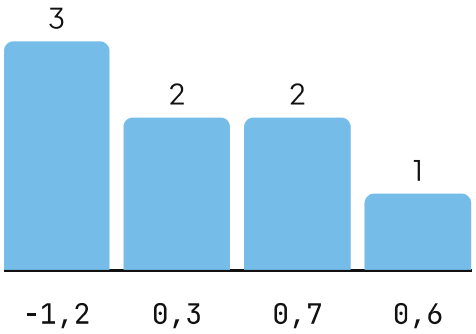
Construção por histogramas

1 · VALORES ORIGINAIS

```
1,2 g = -0,4
1,9 g = -0,3
2,4 g = -0,5
3,1 g = 0,1
3,3 g = 0,2
4,8 g = 0,4
5,0 g = 0,3
6,7 g = 0,6
```

7 cortes possíveis

2 · HISTOGRAMA



soma de g por bin · bins [1; 2,5) [2,5; 4) [4; 5,5) [5,5; 7]

3 · CANDIDATOS

`x < 2,5 → 0,97`

`x < 4,0 → 0,70`

`x < 5,5 → 0,35`

Ganho proporcional: G_L^2/H_L + G_R^2/H_R - G^2/H, com $h = 1$, $\lambda = 0$

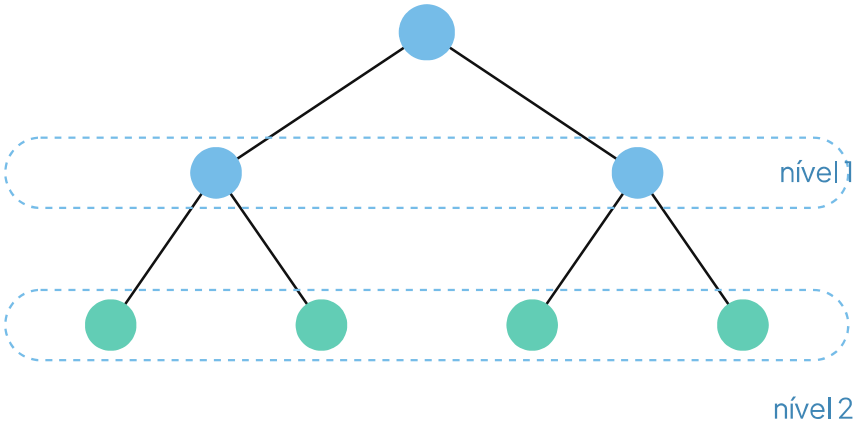
4 · MELHOR DIVISÃO

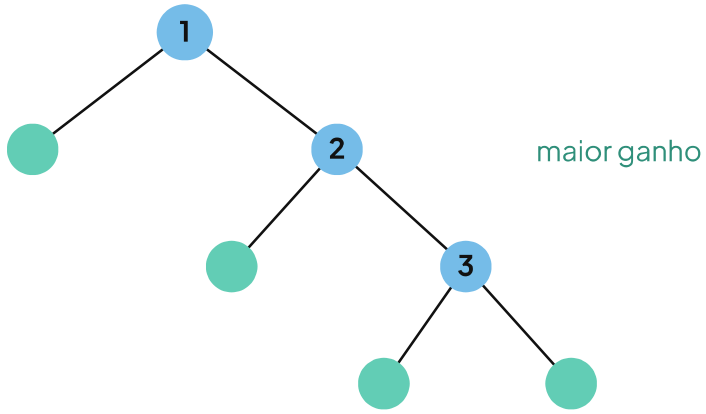
$x < 2{,}5$

3 candidatos em vez de 7. Em dados reais, até 255 bins por característica em vez de milhares de valores.

05 · LIGHTGBM

Crescimento level-wise e leaf-wise





Level-wise

Expande todos os nós de um nível antes de descer. Árvores equilibradas.

Leaf-wise

Expande a folha de maior ganho. Reduz a perda rápido, mas pode criar ramos profundos em dados pequenos.

05 · LIGHTGBM

Hiperparâmetros do LightGBM

`n_estimators`	máximo de árvores
`learning_rate`	η; menor exige mais árvores
`num_leaves`	**máximo de folhas; principal controle de complexidade**
`max_depth`	trava de profundidade; −1 = sem limite
`min_child_samples`	mínimo de amostras por folha
`subsample`	fração de linhas (bagging_fraction)
`subsample_freq`	frequência da amostragem; 0 desativa

`colsample_bytree`	fração de colunas por árvore
`reg_alpha`	L1 nos pesos das folhas
`reg_lambda`	L2 nos pesos das folhas
`objective`	perda; "binary" para binária
`metric`	métrica monitorada; ou eval_metric no fit
`early_stopping`	callback passado ao fit

Aumentar `num_leaves` e `max_depth` aumenta capacidade e risco de overfitting; aumentar `min_child_samples` e a regularização reduz esse risco.

05 · LIGHTGBM · CÓDIGO

num_leaves, max_depth e subsample_freq

Coerência entre folhas e profundidade

$$\texttt{num_leaves} \leq 2^{\texttt{max_depth}}$$

Com `max_depth=5`, o máximo são 32 folhas; 31 é coerente.

Armadilha

`subsample=0.8` sozinho não amostra nada: o padrão `subsample_freq=0` desativa a amostragem.

`subsample_freq=1` sorteia 80% das linhas, sem reposição, a cada iteração.

```python
from lightgbm import LGBMClassifier

modelo = LGBMClassifier(
    n_estimators=800,
    learning_rate=0.05,
    max_depth=5,
    num_leaves=31,        # 31 ≤ 2**5
    min_child_samples=20,
    subsample=0.8,
    subsample_freq=1,     # ativa a amostragem
    colsample_bytree=0.8,
    reg_lambda=1.0,
    objective="binary",
    random_state=42,
    verbosity=-1,
)
```

05 · LIGHTGBM

Recursos avançados: GOSS e EFB

GRADIENT-BASED ONE-SIDE SAMPLING

GOSS



|gradiente| por amostra · rosa: mantidas · azul: sorteadas · cinza: descartadas

Mantém as amostras de gradiente grande, sorteia parte das demais e compensa seus pesos.

EXCLUSIVE FEATURE BUNDLING

EFB

A	B	C		A+B+C
1	0	0	→	1
0	1	0		2
0	0	1		3

colunas que nunca são não nulas ao mesmo tempo

Agrupa características mutuamente exclusivas em um pacote, reduzindo o número de colunas.

São estratégias internas de eficiência. GOSS não é o bootstrap do Bagging: não há reposição nem votação entre modelos independentes.

06

CatBoost

Características categóricas, estatísticas ordenadas e Ordered Boosting

06 · CATBOOST

O que é CatBoost

Categorical Boosting

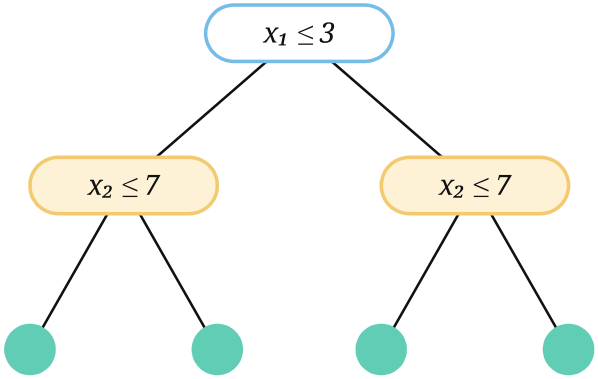
Gradient Boosting desenvolvido com atenção especial ao tratamento de características categóricas.

Categóricas via `cat_features`

Ordered Boosting contra prediction shift

Árvores simétricas no padrão

Regularização com `l2_leaf_reg`



mesma condição em todo o nível

Árvore simétrica (oblivious)

06 · CATBOOST

Problemas de encoding categórico

DADOS ORIGINAIS

Cliente	Plano	Região	Compra
A	Básico	Sul	0
B	Premium	Norte	1
C	Básico	Norte	0
D	Premium	Sul	1

Básico e Premium não têm ordem nem distância numérica.

ONE-HOT

Básico	Premium
1	0
0	1
1	0
0	1

Bom com poucas categorias. Com milhares, gera muitas colunas esparsas.

TARGET ENCODING INGÊNUO

Plano	média
Básico	0,0
Premium	1,0
Básico	0,0
Premium	1,0

Cada linha usa o próprio alvo: a coluna "prevê" a compra perfeitamente no treino. Vazamento.

06 · CATBOOST

Ordered Target Statistics

$$\hat{x}_i = \frac{\Sigma_{\text{anteriores, mesma categoria}}\, y + a \cdot P}{\text{nº de anteriores} + a}$$

P: valor a priori (aqui 0,5)

a: peso do prior (aqui 1)

anteriores: linhas antes de i na permutação

1º na permutação

A · Básico

anteriores Básico: nenhum

`(0 + 0,5)/(0 + 1)`

0,50

2º na permutação

B · Premium

anteriores Premium: nenhum

`(0 + 0,5)/(0 + 1)`

0,50

3º na permutação

C · Básico

anteriores Básico: A (y = 0)

`(0 + 0,5)/(1 + 1)`

0,25

4º na permutação

D · Premium

anteriores Premium: B (y = 1)

`(1 + 0,5)/(1 + 1)`

0,75

Nenhuma linha usa o próprio alvo. O CatBoost usa várias permutações e mecanismos próprios; não é uma média global do alvo.

06 · CATBOOST

Ordered Boosting e prediction shift

Problema O resíduo da amostra i é calculado por um modelo que já viu y_i. Os resíduos de treino ficam deslocados em relação aos de dados novos.

1

Permutar

3 1 4 2 5

Sorteia-se uma ordem para as amostras de treino.

2

Usar só o passado

3 1 4 → r_2

O resíduo da amostra 2 vem de um modelo treinado com 3, 1 e 4.

3

Ajustar a árvore

r_3 r_1 r_4 r_2 r_5

A nova árvore aprende com resíduos que não usaram o próprio alvo.

Reduz o prediction shift; não elimina qualquer vazamento. `boosting_type`: "Ordered" ou "Plain" (padrão depende dos dados e do dispositivo).

06 · CATBOOST · CÓDIGO

Hiperparâmetros do CatBoost

`iterations`	máximo de árvores
`learning_rate`	η; automático se omitido
`depth`	profundidade; 2^d folhas
`l2_leaf_reg`	L2 nos pesos das folhas
`loss_function`	perda otimizada
`eval_metric`	métrica monitorada
`random_seed`	semente
`cat_features`	colunas categóricas (int ou str)
`verbose`	mensagens de treino
`early_stopping_rounds`	paciência; no fit
`allow_writing_files`	False evita catboost_info/

```python
from catboost import CatBoostClassifier

modelo = CatBoostClassifier(
    iterations=800,
    learning_rate=0.05,
    depth=5,
    l2_leaf_reg=3.0,
    loss_function="Logloss",
    eval_metric="AUC",
    random_seed=42,
    verbose=False,
    allow_writing_files=False,
)
modelo.fit(X_train, y_train,
           cat_features=["plano", "regiao"],
           eval_set=(X_val, y_val),
           early_stopping_rounds=40)
```

07

Comparação

O que é comum, o que diferencia e quando considerar cada um

07 · COMPARAÇÃO

XGBoost, LightGBM e CatBoost

	XGBoost	LightGBM	CatBoost
Base	Gradient Boosting	Gradient Boosting	Gradient Boosting
Estratégia marcante	Regularização e informação de segunda ordem	Histogramas e crescimento leaf-wise	Tratamento categórico e estratégias ordenadas
Categóricas	Exigem configuração ou preparação adequada	Suporte próprio para categóricas	Suporte nativo por `cat_features`
Crescimento	Estratégias próprias (depthwise, lossguide)	Leaf-wise na configuração GBDT usual	Árvores simétricas comuns no padrão
Parâmetro marcante	`min_child_weight, gamma`	`num_leaves`	`depth, l2_leaf_reg`

Visão geral, não uma descrição completa de todas as configurações possíveis.

07 · COMPARAÇÃO

Quando considerar cada um

Tabular numérico

Os três são candidatos; o ajuste pesa mais que a biblioteca.

Muitas linhas

LightGBM e XGBoost com `hist` escalam bem.

Muitas categorias

CatBoost reduz preparação e risco de vazamento no encoding.

Velocidade

Meça no seu hardware; não assuma.

Regularização

Controles explícitos nos três, com nomes e efeitos próprios.

Recursos limitados

Histogramas e menos bins reduzem memória.

Dados pequenos

Restrinja leaf-wise; use early stopping e validação cruzada.

Desbalanceados

Pesos de classe, métricas adequadas e ajuste do limiar.

- Não existe vencedor universal. A escolha vem de validação adequada e das características do problema.

07 · COMPARAÇÃO

Limitações do Boosting

Limitação	Estratégia de controle
Sensibilidade a hiperparâmetros	busca sistemática com validação
Tempo de ajuste	early stopping, busca aleatória
Overfitting	árvores menores, regularização, subamostragem, early stopping
Dados ruidosos	η menor, folhas com mais amostras, perdas robustas
Valores ausentes	tratamento interno existe, mas investigue a causa
Custo computacional	menos árvores via early stopping, histogramas
Interpretabilidade	permutation importance, SHAP, dependência parcial
Importância ≠ causalidade	tratar importância como associação preditiva

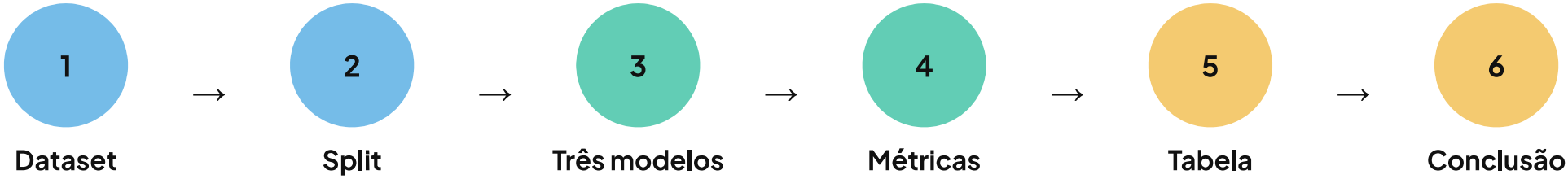
08

Benchmark guiado

Três algoritmos no mesmo split, a partir de um notebook vazio

08 · BENCHMARK

Plano do experimento



Por que `make_classification`

Sem download · binário · totalmente numérico · reproduzível · mesma matriz para os três modelos.

Fora do núcleo em sala

Importância por permutação e ajuste de hiperparâmetros: continuação guiada e desafio. Categóricas: exemplo complementar separado.

08 · ETAPA 1

Bibliotecas e versões

```
# trecho; ver notebookimport time, importlib.metadata
import numpy as np, pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (accuracy_score, ...)
from xgboost import XGBClassifier
from lightgbm import (LGBMClassifier,
    early_stopping, log_evaluation)
from catboost import CatBoostClassifier

for p in ["numpy", "pandas", "scikit-learn",
          "xgboost", "lightgbm", "catboost"]:
    print(p, importlib.metadata.version(p))
```

Instalação local

```
python -m pip install numpy pandas matplotlib
scikit-learn xgboost lightgbm catboost
```

Google Colab

```
%pip install -q xgboost lightgbm catboost
```

Versões instaladas

PREENCHER APÓS RUN ALL

08 · ETAPA 2

Criação do dataset

```python
X, y = make_classification(
    n_samples=2400,
    n_features=16,
    n_informative=8,
    n_redundant=4,
    n_repeated=0,
    n_clusters_per_class=2,
    weights=[0.55, 0.45],
    flip_y=0.03,
    class_sep=1.0,
    random_state=42
)
print("Formato de X:", X.shape)
print("Formato de y:", y.shape)
print("Classes:", np.unique(y, return_counts=True))
```

`n_samples`	2400 linhas
`n_features`	16 colunas no total
`n_informative`	8 colunas com sinal
`n_redundant`	4 combinações lineares das informativas
`restantes`	4 colunas de ruído
`n_clusters_per_class`	2 grupos por classe
`weights`	≈ 55% classe 0, 45% classe 1
`flip_y`	≈ 3% de rótulos trocados
`class_sep`	separação moderada
`random_state`	reprodutibilidade

08 · ETAPA 3

Exploração dos dados

```
colunas = [f"feature_{i:02d}" for i in range(1, 17)]
df = pd.DataFrame(X, columns=colunas)
df["target"] = y

display(df.head())
print(df.shape)
print(df.dtypes.value_counts())
print(df["target"].value_counts(normalize=True))
display(df.describe().T)

df["target"].value_counts().sort_index().plot(
    kind="bar", color=["#75BCE8", "#62CDB5"])
```

Distribuição dos rótulos

PREENCHER APÓS RUN ALL

Conferir: 2400 × 17, tipos numéricos, proporção próxima de 55/45.

08 · ETAPA 4

Treino, validação e teste



ajusta as árvores

early stopping e escolhas

avaliação final, uma vez

```
X_df = df[colunas]
X_temp, X_test, y_temp, y_test = train_test_split(
    X_df, y, test_size=0.20, stratify=y, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(
    X_temp, y_temp, test_size=0.25,
    stratify=y_temp, random_state=42)
```

0,25 × 80% = 20% do total para validação.

O teste não escolhe hiperparâmetros nem interrompe o treinamento.

Os três modelos usam exatamente as mesmas amostras.

08 · ETAPA 5

Baseline

```
baseline = DummyClassifier(strategy="prior")
baseline.fit(X_train, y_train)

pred_base = baseline.predict(X_test)
acc_base = accuracy_score(y_test, pred_base)
print(f"Acurácia do baseline: {acc_base:.4f}")
```



Prevê sempre a classe majoritária e usa as proporções do treino como probabilidade.

08 · ETAPAS 6 E 7

Configuração e treino do XGBoost

```
modelo_xgb = XGBClassifier(
    n_estimators=800,
    max_depth=5, learning_rate=0.05,
    objective="binary:logistic",
    eval_metric="auc",
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0, random_state=42,
    tree_method="hist", n_jobs=-1)
# early stopping onde a versão exige
if versao_xgb ⩾ (1, 6):
    modelo_xgb.set_params(
        early_stopping_rounds=40)
    fit_extra_xgb = {}
else:
    fit_extra_xgb = {"early_stopping_rounds": 40,
                     "eval_metric": "auc"}
```

```
inicio = time.perf_counter()
modelo_xgb.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    verbose=False, **fit_extra_xgb)
tempo_xgb = time.perf_counter() - inicio

arvores_xgb = modelo_xgb.best_iteration + 1
```

tempo medido
PREENCHER APÓS RUN ALL

melhor iteração
PREENCHER APÓS RUN ALL

Configuração e treino do LightGBM

```
modelo_lgbm = LGBMClassifier(
    n_estimators=800,
    learning_rate=0.05,
    max_depth=5,
    num_leaves=31,       # ≤ 2**5
    subsample=0.8,
    subsample_freq=1,    # ativa
    colsample_bytree=0.8,
    reg_lambda=1.0,
    objective="binary",
    metric="auc",        # só AUC
    random_state=42,
    n_jobs=-1,
    verbosity=-1)
```

```
inicio = time.perf_counter()
modelo_lgbm.fit(
    X_train, y_train,
    eval_set=[(X_val, y_val)],
    eval_metric="auc",
    callbacks=[
        early_stopping(40, verbose=False),
        log_evaluation(0)])
tempo_lgbm = time.perf_counter() - inicio
arvores_lgbm = modelo_lgbm.best_iteration_
```

tempo medido
PREENCHER APÓS RUN ALL

melhor iteração
PREENCHER APÓS RUN ALL

08 · ETAPAS 10 E 11

Configuração e treino do CatBoost

```
modelo_cat = CatBoostClassifier(
    iterations=800,
    depth=5,
    learning_rate=0.05,
    loss_function="Logloss",
    eval_metric="AUC",
    l2_leaf_reg=3.0,
    random_seed=42,
    verbose=False,
    allow_writing_files=False,
    thread_count=-1)
```

```
inicio = time.perf_counter()
modelo_cat.fit(
    X_train, y_train,
    eval_set=(X_val, y_val),
    early_stopping_rounds=40,
    verbose=False)
tempo_cat = time.perf_counter() - inicio
arvores_cat = modelo_cat.tree_count_
```

tempo medido
PREENCHER APÓS RUN ALL

melhor iteração
PREENCHER APÓS RUN ALL

`eval_set` é uma tupla e a paciência vai no `fit`: APIs diferentes, documentadas separadamente.

08 · ETAPA 12

Conferência da comparação justa

- ✓ Mesmo dataset
- ✓ Mesmos `X_train`, `X_val`, `X_test`
- ✓ Mesmos rótulos
- ✓ Mesma semente de referência (42)
- ✓ Mesma métrica de early stopping (AUC)
- ✓ Mesmo limiar de classificação (0,5)
- ✓ Teste separado do processo de seleção

O QUE NÃO É IDÊNTICO

XGBoost	até 32 folhas, depthwise, λ = 1
LightGBM	até 31 folhas, leaf-wise, λ = 1
CatBoost	32 folhas simétricas, l2 = 3

Benchmark de configurações documentadas sob os mesmos dados e protocolo, não prova universal de superioridade.

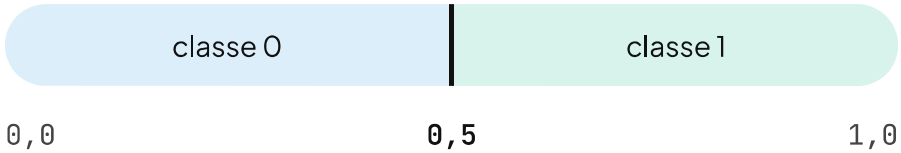
08 · ETAPAS 13 E 14

Probabilidades e limiar comum

```python
def probabilidade_classe_positiva(modelo, X):
    probabilidades = modelo.predict_proba(X)
    indice_positivo = list(modelo.classes_).index(1)
    return probabilidades[:, indice_positivo]

LIMIAR = 0.5
modelos = {"XGBoost": modelo_xgb,
           "LightGBM": modelo_lgbm,
           "CatBoost": modelo_cat}

proba_teste, pred_teste = {}, {}
for nome, m in modelos.items():
    p = probabilidade_classe_positiva(m, X_test)
    proba_teste[nome] = p
    pred_teste[nome] = (p ⩾ LIMIAR).astype(int)
```



Limiar ↑ menos positivos: precisão tende a subir, recall a cair.

Limiar ↓ mais positivos: recall tende a subir, precisão a cair.

0,51 e 0,99 viram a mesma classe, com confiança muito diferente.

08 · ETAPA 15

Métricas comuns

```
def calcular_metricas(nome, y_true, proba, tempo=np.nan):
    pred = (proba ⩾ LIMIAR).astype(int)
    return {
        "Modelo": nome,
        "Acurácia": accuracy_score(y_true, pred),
        "Precisão": precision_score(y_true, pred,
                                    zero_division=0),
        "Recall": recall_score(y_true, pred,
                               zero_division=0),
        "F1": f1_score(y_true, pred, zero_division=0),
        "ROC AUC": roc_auc_score(y_true, proba),
        "Log Loss": log_loss(y_true, proba),
        "Tempo (s)": tempo}
```

Acurácia	proporção de acertos
Precisão	dos previstos 1, quantos são 1
Recall	dos reais 1, quantos foram encontrados
F1	média harmônica de precisão e recall
ROC AUC	qualidade da ordenação; não depende do limiar
Log Loss	qualidade das probabilidades; menor é melhor

As quatro primeiras usam o limiar de 0,5.

08 · ETAPA 16

Matrizes de confusão

XGBoost

VN PREENCHER	FP PREENCHER
FN PREENCHER	VP PREENCHER

LightGBM

VN PREENCHER	FP PREENCHER
FN PREENCHER	VP PREENCHER

CatBoost

VN PREENCHER	FP PREENCHER
FN PREENCHER	VP PREENCHER

```
cm = confusion_matrix(y_test, pred_teste[nome],
                      labels=[0, 1])
ConfusionMatrixDisplay(cm, display_labels=[0, 1]
                       ).plot(ax=ax)
```

Linhas: classe real · colunas: classe prevista · mesma ordem [0, 1] nas três.

08 · ETAPA 17

Tabela comparativa no teste

Modelo	Acurácia	Precisão	Recall	F1	ROC AUC	Log Loss	Tempo (s)
Baseline	—*	—*	—*	—*	—*	—*	—*
XGBoost	—*	—*	—*	—*	—*	—*	—*
LightGBM	—*	—*	—*	—*	—*	—*	—*
CatBoost	—*	—*	—*	—*	—*	—*	—*

—* PREENCHER APÓS RUN ALL com os valores do notebook. Versões das bibliotecas registradas abaixo da tabela.

O tempo varia com hardware, sistema operacional, paralelismo e configuração; vale para esta máquina e esta execução.

08 · ETAPAS 18 E 19

Tempo de treino e importância por permutação

Barras: tempo medido (s)

PREENCHER APÓS RUN ALL

Ler junto com o número efetivo de árvores após o early stopping.

```python
def auc_validacao(modelo, X, y):
    proba = probabilidade_classe_positiva(modelo, X)
    return roc_auc_score(y, proba)

for nome, modelo in modelos.items():
    resultado = permutation_importance(
        modelo, X_val, y_val,
        scoring=auc_validacao,
        n_repeats=5, random_state=42)
    importancias[nome] = resultado.importances_mean
```

Importância de característica não significa causalidade. Importâncias nativas têm definições diferentes em cada biblioteca.

08 · ETAPA 20

Conclusão do benchmark

Melhor ROC AUC	PREENCHER
Melhor F1	PREENCHER
Menor Log Loss	PREENCHER
Menor tempo	PREENCHER
Maior gap treino – validação	PREENCHER
Importâncias concordam?	PREENCHER

LIMITAÇÕES DO EXPERIMENTO

Um dataset sintético

Um split e uma semente

Hiperparâmetros não ajustados nem equivalentes

Tempo de uma única execução

Diferenças pequenas podem mudar com outra semente

08 · EXEMPLO COMPLEMENTAR

CatBoost com características categóricas

Idade	Plano	Região	Compra
34	Básico	Sul	0
51	Premium	Norte	1
28	Padrão	Leste	0
...	...	...	...

Colunas categóricas como strings ou inteiros, sem NaN.

XGBoost e LightGBM têm caminhos próprios, que dependem da versão e da configuração.

```
colunas_cat = ["plano", "regiao"]

modelo_cat_demo = CatBoostClassifier(
    iterations=300, depth=4,
    learning_rate=0.05,
    loss_function="Logloss",
    eval_metric="AUC", random_seed=42,
    verbose=False, allow_writing_files=False)

modelo_cat_demo.fit(
    Xc_train, yc_train,
    cat_features=colunas_cat,
    eval_set=(Xc_val, yc_val),
    early_stopping_rounds=30)
```

REVISÃO · 1/2

Erros comuns

Erro	Por que está errado	Correção
Bagging = Boosting	um é independente, o outro sequencial	Bagging combina; Boosting soma correções
Árvores do Boosting independentes	cada árvore usa a previsão acumulada	a etapa m depende de $1 \dots m-1$
Todo gradiente negativo é resíduo	coincidem só na perda quadrática	usar o termo pseudo-resíduo
As três bibliotecas são iguais	diferem em árvores, regularização, categóricas	base comum, implementações distintas
Learning rate como em redes neurais	aqui encolhe árvores somadas	ler no contexto de $F_m = F_{m-1} + \eta h_m$
n_estimators = profundidade	quantidade ≠ tamanho das árvores	ajustar os dois separadamente
gamma do XGBoost = gamma da SVM	penalização por folha vs. alcance do kernel	ganho mínimo para dividir
Ignorar num_leaves	é o principal controle no leaf-wise	num_leaves ≤ 2^max_depth
subsample sem subsample_freq	freq = 0 desativa a amostragem	subsample_freq=1

REVISÃO · 2/2

Erros comuns

Erro	Por que está errado	Correção
CatBoost elimina todo vazamento	reduz prediction shift, não vazamento do dataset	auditar colunas e protocolo
Teste no early stopping	o teste passa a influenciar o modelo	parar pela validação
Métricas de splits diferentes	o split muda o resultado	mesmas amostras para todos
Tempos em condições diferentes	hardware e núcleos dominam o tempo	mesma máquina e paralelismo
Ignorar versões	APIs e padrões mudam	registrar versões no notebook
Métricas inventadas	não são reproduzíveis	Restart Kernel → Run All
ROC AUC = acurácia	AUC mede ordenação, sem limiar	reportar as duas, com significado
Probabilidade = classe	a classe depende do limiar	predict_proba + limiar explícito
Importâncias nativas comparadas	gain, split e afins têm definições diferentes	permutation importance comum

QUIZ · 1/2

Quiz

1. O que é Boosting?
2. Qual a diferença entre Bagging e Boosting?
3. O que significa aprendizado sequencial?
4. O que são resíduos?
5. O que são pseudo-resíduos?
6. O que é Gradient Boosting?
7. O que controla o learning rate?
8. O que representa `n_estimators`?
9. Como funciona o XGBoost?
10. O que é regularização no XGBoost?

QUIZ · 2/2

Quiz

11 O que representam gradiente e Hessiana?

12 Como funciona o crescimento leaf-wise?

13 O que significa `num_leaves`?

14 Por que o LightGBM utiliza histogramas?

15 O que diferencia o CatBoost?

16 O que é Ordered Boosting?

17 O que são características categóricas?

18 O que é Early Stopping?

19 Como comparar três algoritmos de maneira justa?

20 Por que não existe um vencedor universal?

DESAFIO FINAL

Investigando o Boosting

Preparar

1 · criar ou carregar outro dataset

2 · explorar os dados

3 · dividir em treino, validação e teste

Treinar

4 · XGBoost

5 · LightGBM

6 · CatBoost

7 · early stopping na validação

13 · dois hiperparâmetros por biblioteca

Analisar

8 · mesmas métricas

9 · matrizes de confusão

10 · tempo

11–12 · comparar e interpretar

14 · conclusão escrita

Entregas

Notebook executável

Tabela comparativa

Gráficos e métricas

Configurações utilizadas

Versões das bibliotecas

Conclusão técnica

Síntese da aula

Bagging vs. Boosting → Resíduos → Gradientes → Gradient Boosting → Learning rate → XGBoost → LightGBM → CatBoost → Benchmark

MATERIAL
Notebook do exercício guiado

MATERIAL
Desafio com TODOs

MATERIAL
Checklist de aprendizagem

PRÓXIMA AULA
Ajuste fino de hiperparâmetros